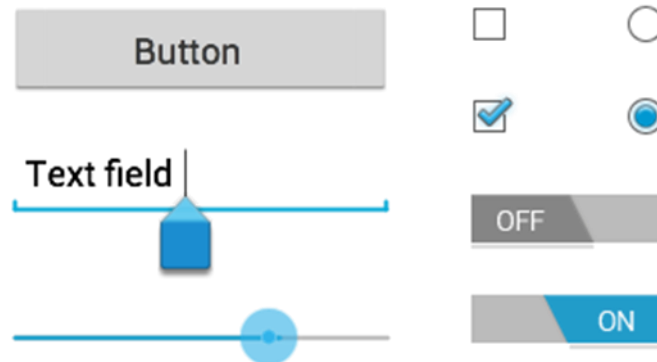


2.1 User Interface

Layouts, Components and Input Events



Objectives

- Explain the overview of User Interface
- Create UI containing common UI components
- Create and use style and theme

Basics of Layout

Foundational layout elements in Compose:

1) Core building blocks:

Row, Column, Box

2) Control layout behaviors (padding, size, alignment):

Modifier

3) Managing space:

Divider

Compose UI is built by **nesting simple layouts declaratively**

ViewGroup – Static Content

1. Row & Column



Organizes its children into a single horizontal (Row) or vertical (Column).

Creates a scrollbar if the length of the window exceeds the length of the screen.

2. Constrain Layout.



Enables you to specify the location of child objects relative to each other

*Has been replaced by the Constraint Layout

3. Web Layout



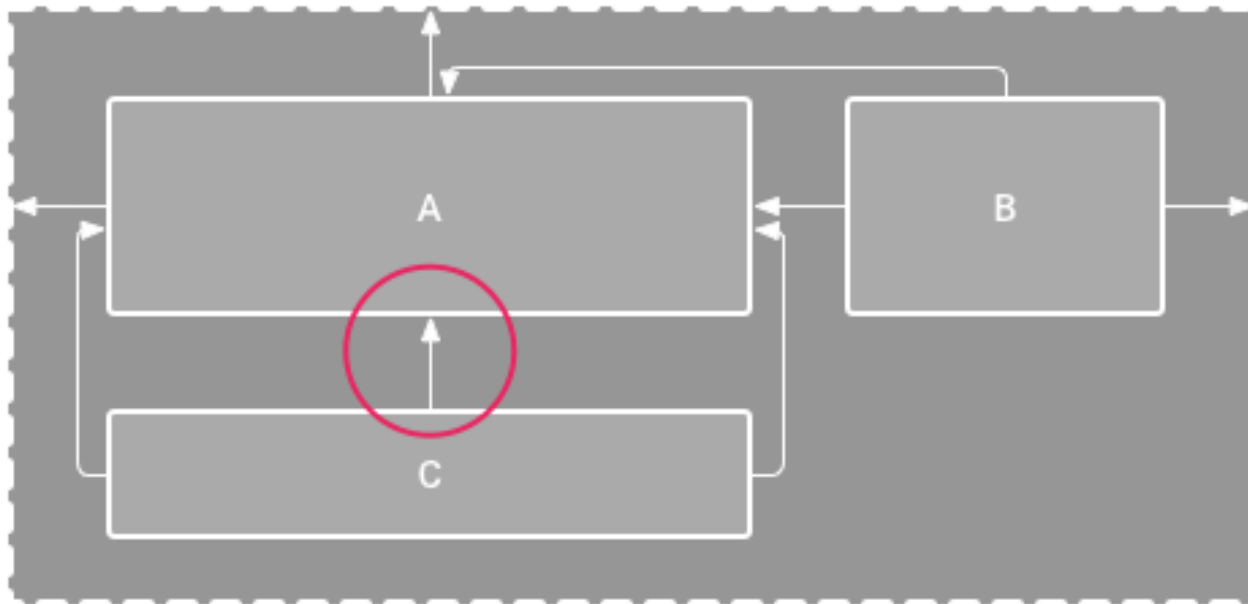
Displays web pages.

Constraint Layout

- Creates large and complex layouts with a flat view hierarchy
- Compatible with Android 2.3 (API level 9) and higher

Constraint Layout

- You must add at least one horizontal and one vertical constraint for the view



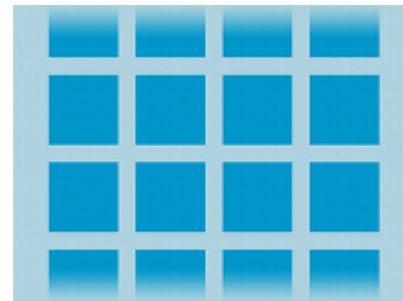
ViewGroup – Dynamic Content

4. Lazy List



Displays a scrolling single column list.

5. Lazy Grid



Displays a scrolling grid of columns and rows

Lazy List

- Lazy Row or Lazy Column do not require adapters like traditional Android RecyclerView

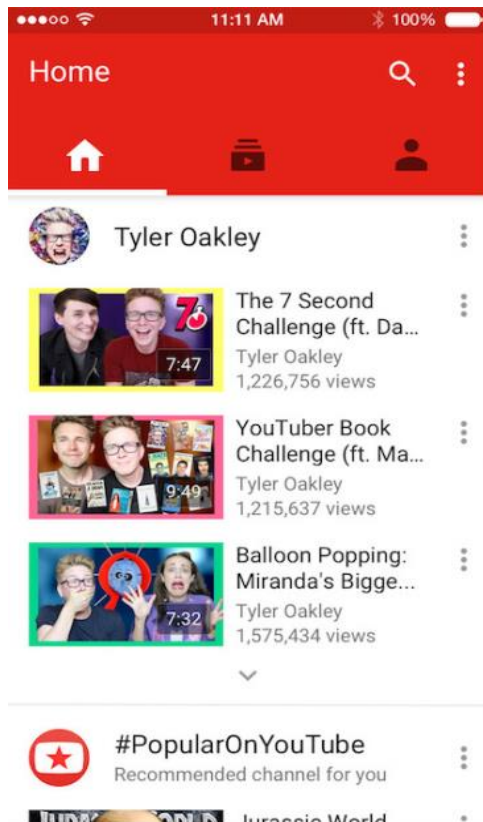
```
val items = listOf("Apple", "Banana", "Cherry")
```

```
LazyColumn {  
    items(items) { item ->  
        Text(text = item)  
    }  
}
```

Question?

- Identify a suitable Layout for each of the following apps:

A



B



Improving Layout Performance in traditional Android

1. Optimizing Layout hierarchies
2. Re-using Layouts with `<include>` tag
3. Loading Views on demand

How Jetpack compose handle Performance issues?

1. Optimizing Layout hierarchies

- In Jetpack compose, there is no more traditional View hierarchy
- Compose uses a flat tree of Composable and only recompose the parts that change.
- Compose layouts are lightweight and designed for performance

How Jetpack compose handle Performance issues?

2. Re-using Layouts with `<include>` tag
 - Instead of using `<include>` tag, Jetpack compose uses composable functions
 - Best practice for Jetpack Compose is to break UI into small reusable Composable.

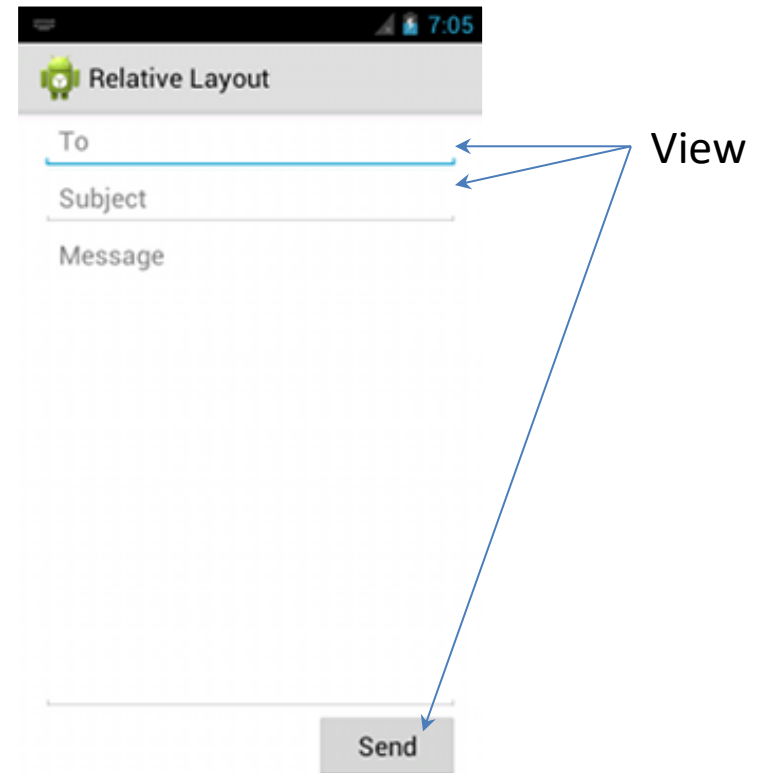
How Jetpack compose handle Performance issues?

3. Loading Views on demand

- Jetpack Compose handles on-demand rendering natively with components like LazyColumn and smart recomposition.
- Only visible items are composed and drawn

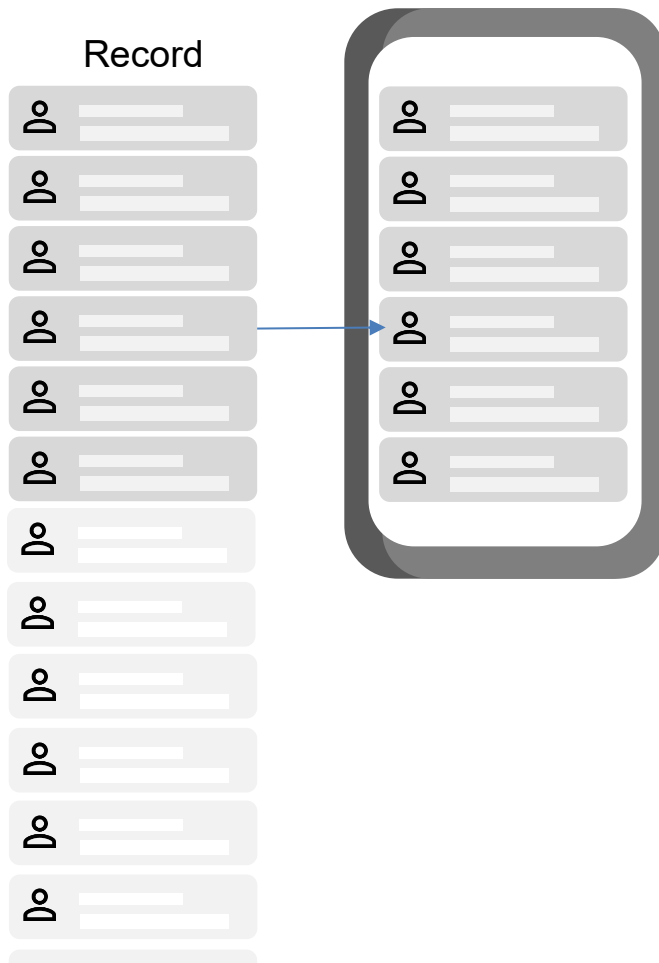
View

- Also called widget
- E.g. Button and TextView



ListView

All records are loaded to the ListView

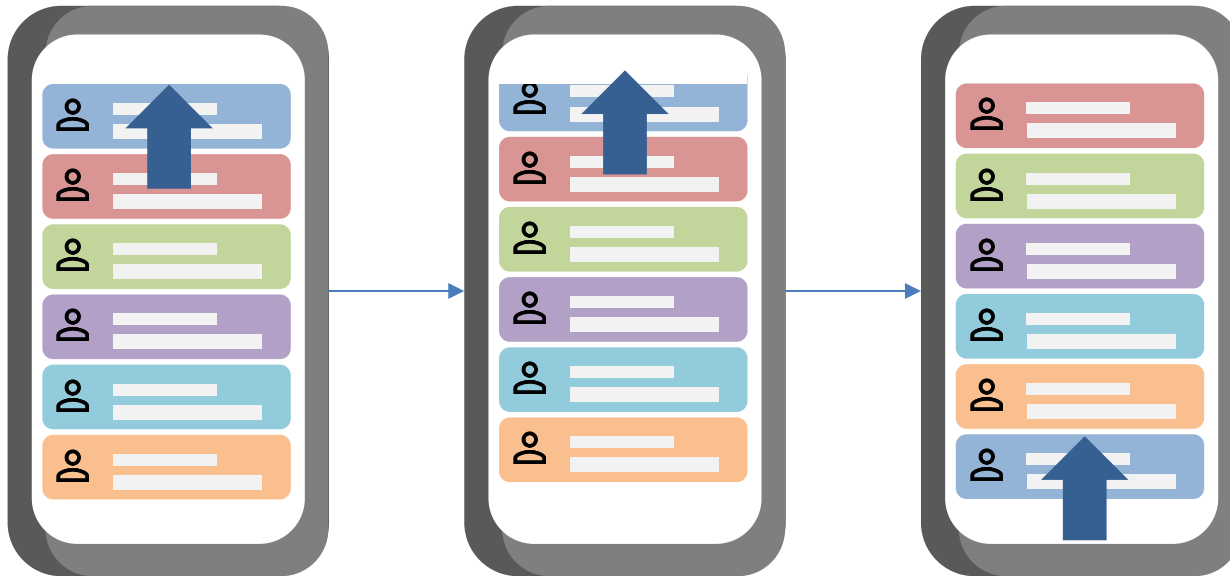


LazyList

Load records that are visible to the user using ViewHolder



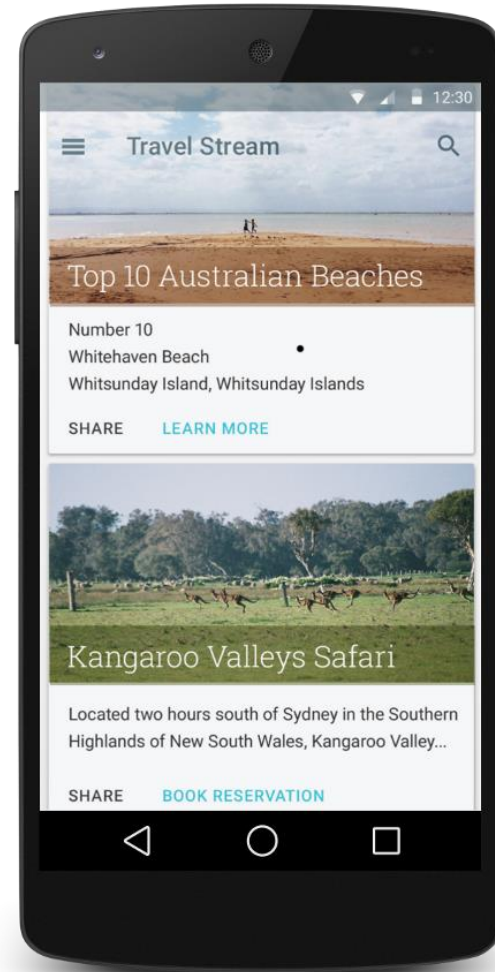
Recycler View



When an item scrolls off the screen, the item's views are recycled. That means the item is filled with new content that scrolls onto the screen.

Card-based Layout

- Allows app to show information inside cards
- Provides a consistent look across different platform

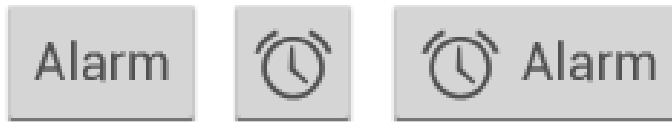


UI Components

- 1.Button
- 2.Text Field
- 3.Check Box
- 4.Radio Button
- 5.Toggle
- 6.Spinner

Button

- Two types:
 - Button class : text and an icon
 - ImageButton class: icon



```
@Composable  
fun MySimpleButton() {
```

```
    Button(onClick = {  
        // Handle button click  
        println("Button clicked!")  
    }) {
```

```
        Text("Click Me")
```

```
    }
```

```
}
```

← Onclick Events handling thru
Lambda expression

Button- Modifier and Styling

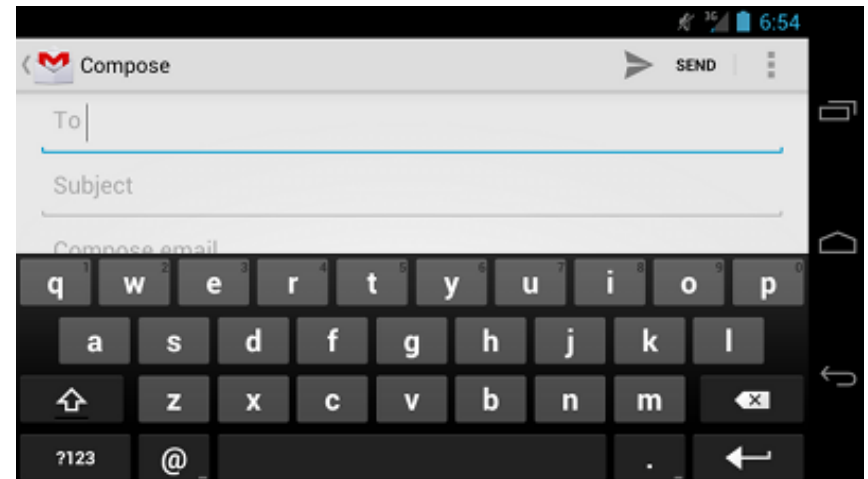
```
@Composable
fun StyledButton() {
    Button(
        onClick = { println("Styled button clicked") },
        modifier = Modifier
            .padding(16.dp)
            .fillMaxWidth(),
        colors = ButtonDefaults.buttonColors(
            containerColor = Color.Blue,
            contentColor = Color.White
        )
    ) {
        Text("Styled Button")
    }
}
```

Button- Icon

```
@Composable
fun IconButtonSample() {
    Button(onClick = { /* do something */ }) {
        Icon(Icons.Default.Favorite, contentDescription =
"Favorite")
        Spacer(modifier = Modifier.width(8.dp))
        Text("Like")
    }
}
```

Text Fields

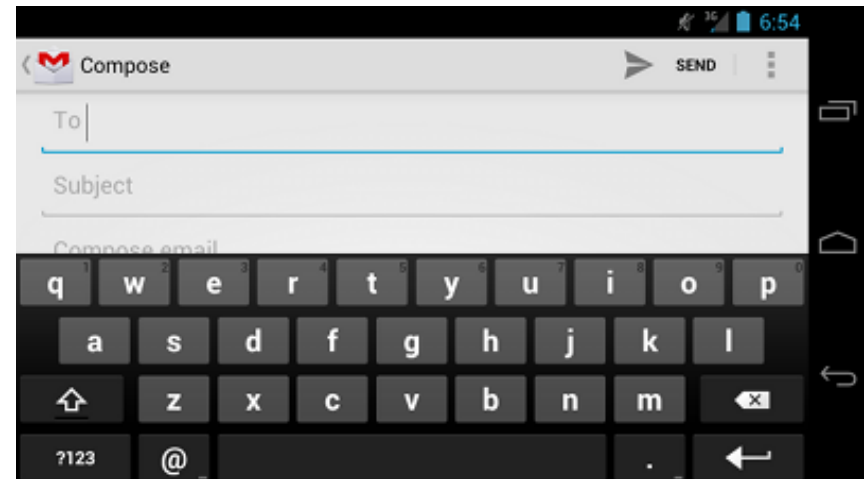
- You may specify the keyboard type in the layout with the keyboardType attribute



Text Fields

```
@Composable
fun EmailInputField() {
    var email by remember { mutableStateOf("") }

    TextField(
        value = email,
        onValueChange = { email = it },
        label = { Text("Email Address") },
        leadingIcon = { Icon(Icons.Default.Email,
contentDescription = null) },
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Email,
            imeAction = ImeAction.Done
        ),
        singleLine = true,
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
    )
}
```



Text Fields

"text"

Normal text keyboard.

"textEmailAddress"

Normal text keyboard with the @ character.

"textUri"

Normal text keyboard with the / character.

"number"

Basic number keypad.

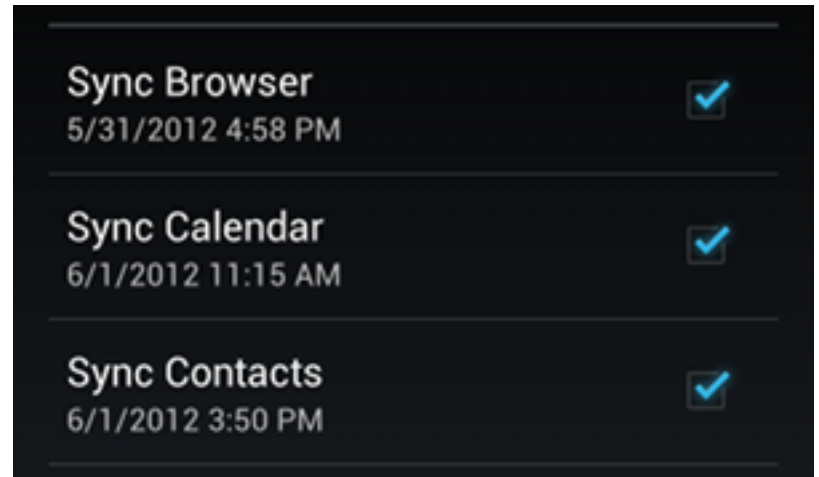
"phone"

Phone-style keypad.



Check Box

- Checkboxes allow the user to select one or more options from a set.
- Present each checkbox option in a vertical list.



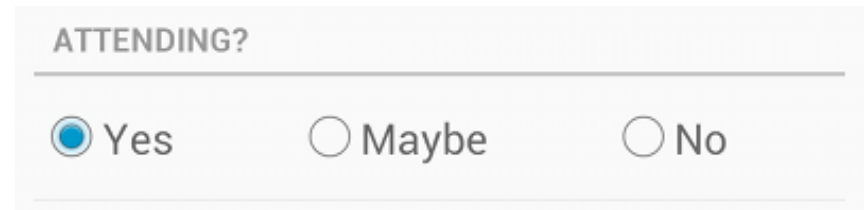
Check Box

```
@Composable
fun SimpleCheckbox() {
    var isChecked by remember { mutableStateOf(false) }

    Row(verticalAlignment = Alignment.CenterVertically)
    {
        Checkbox(
            checked = isChecked,
            onCheckedChange = { isChecked = it }
        )
        Text(text = if (isChecked) "Checked" else
"Unchecked")
    }
}
```

Radio Button

- Allows user to select one option from a set.
- Use it for optional sets that are mutually exclusive if you think that the user needs to see all available options side-by-side.



ATTENDING?

☒ Yes ☐ Maybe ☐ No

<https://developer.android.com/guide/topics/ui/controls/radiobutton>

Radio Button

```
@Composable
fun SimpleRadioButton() {
    var isSelected by remember { mutableStateOf(false) }

    Row(
        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.clickable { isSelected =
!isSelected }
    ){
        RadioButton(
            selected = isSelected,
            onClick = null // Let Row handle clicks
        )
        Spacer(modifier = Modifier.width(8.dp))
        Text("Option A")
    }
}
```

Group Radio Button

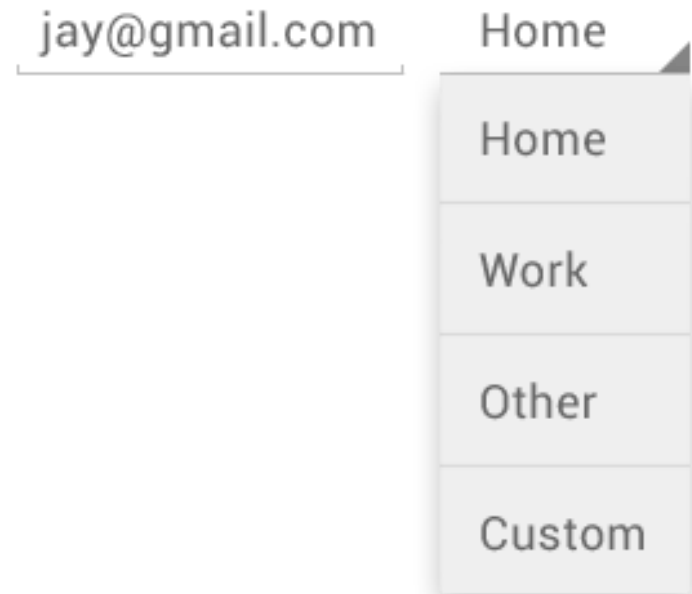
```
@Composable
fun RadioGroupExample() {
    val options = listOf("Male", "Female", "Other")
    var selectedOption by remember { mutableStateOf(options[0]) }

    Column(modifier = Modifier.padding(16.dp)) {
        Text("Select Gender", style = MaterialTheme.typography.titleMedium)

        options.forEach { option ->
            Row(
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier
                    .fillMaxWidth()
                    .clickable { selectedOption = option }
                    .padding(vertical = 8.dp)
            ) {
                RadioButton(
                    selected = (option == selectedOption),
                    onClick = null
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text(option)
            }
        }
    }
}
```

Spinners (DropDownMenu)

- Provide a quick way to select one value from a set
- By default, a spinner shows its currently selected value.
- Displays a dropdown menu with all other available values



Spinners

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SpinnerExample() {
    val options = listOf("Apple", "Banana", "Cherry")
    var expanded by remember { mutableStateOf(false) }
    var selectedOption by remember { mutableStateOf(options[0]) }

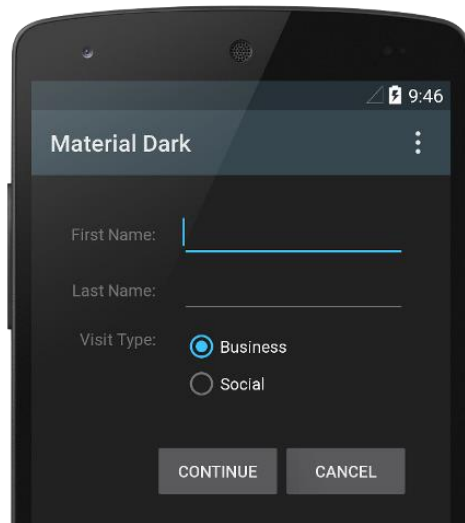
    ExposedDropdownMenuBox(
        expanded = expanded,
        onExpandedChange = { expanded = !expanded }
    ) {
        TextField(
            value = selectedOption,
            onValueChange = {},
            readOnly = true,
            label = { Text("Select Fruit") },
            trailingIcon = {
                ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded)
            },
            modifier = Modifier.menuAnchor()
        )
    }
}
```

Spinners

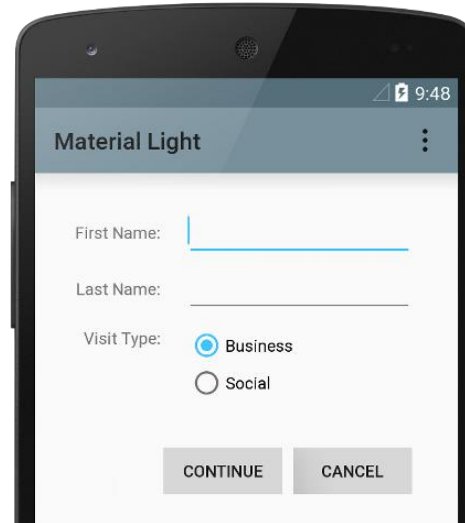
```
ExposedDropdownMenu(  
    expanded = expanded,  
    onDismissRequest = { expanded = false }  
) {  
    options.forEach { item ->  
        DropdownMenuItem(  
            text = { Text(item) },  
            onClick = {  
                selectedOption = item  
                expanded = false  
            }  
        )  
    }  
}  
}
```

Style and Theme

- Use the Android's style and theme resources to change appearance of action bar.



Dark material theme



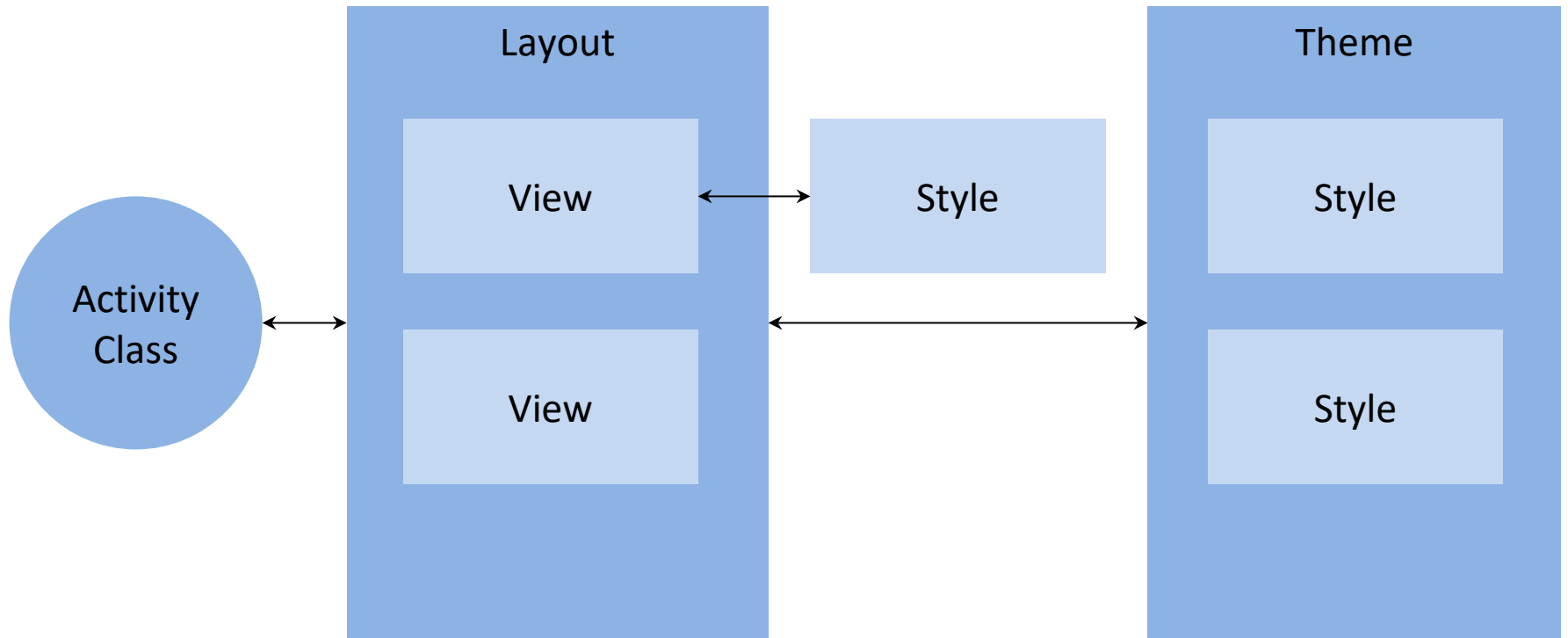
Light material theme

Use an Android Theme

- You can apply these themes to your entire app or to individual activities by declaring them in `MaterialTheme`

```
MaterialTheme(  
    colorScheme = myColorScheme,  
    typography = myTypography,  
    shapes = myShapes  
) {  
    // Your Composables here  
}
```

Style vs. Theme



Dark Theme

- Available in Android 10 (API level 29) and higher
- Benefits
 - Reduces power usage
 - Improves visibility for users
 - Easier to use in a low-light environment

Dark Theme

- Support Dark theme in your app by:
 - Set your app's theme to inherit from a DayNight theme

```
<style name="AppTheme" parent="Theme.AppCompat.DayNight">
```

Or

```
<style name="AppTheme" parent="Theme.MaterialComponents.DayNight">
```